UNIVERSITÉ ALIOUNE DIOP DE BAMBEY
UFR Sciences & Technologies | Master 2 Big Data & Intelligence Artificielle
Année universitaire 2025-2026

PROJET TUTORÉ — CODES SOURCES COMPLETS

Teranga-SN

Intelligence Touristique & Économie Numérique

Tourisme & E-commerce Local

Durée	4 semaines
Équipe	2 étudiants
Domaine	Tourisme & E-commerce Local
Standard	M2 Approche Industrie | New Deal Technologique Sénégal
Enseignant	Mr Ahmed Ben Sidy Bouya SEYE — Senior Big Data & AI Engineer | Groupe Sonatel

NiFi • Kafka • Spark Streaming • Pandera • HBase • Hive • Airflow • MLlib • Privacy by Design

1. Contexte et Objectifs

Teranga signifie « hospitalité » en Wolof. Le Sénégal vise 3 millions de touristes/an à l'horizon 2035. Ce projet construit un système d'analyse des flux touristiques et du sentiment des visiteurs sur les plateformes numériques, couplé à une analyse du commerce électronique local (Expat-Dakar, Jumia Sénégal) pour optimiser les campagnes de promotion et détecter les opportunités économiques.

> ℹ️ **New Deal Technologique Sénégal**
> Ce projet s'inscrit dans la Stratégie Sénégal Numérique 2025-2035 et le Plan Sénégal Émergent. L'objectif est de former des ingénieurs Big Data capables de construire des solutions Data-Driven directement applicables aux défis de développement du Sénégal.

1.1 Objectifs Techniques

#	Composant	Description
1	**Ingestion multi-sources**	avis TripAdvisor/Google, données DGTT (arrivées), flux Jumia
2	**Analyse de sentiment des avis touristiques (FR/EN/WO) par région et type d'activité**	Analyse de sentiment des avis touristiques (FR/EN/WO) par région et type d'activité
3	**Prédiction des pics de fréquentation touristique par destination**	Prédiction des pics de fréquentation touristique par destination
4	**Privacy by Design**	anonymisation des données transactionnelles e-commerce
5	**Dashboard DGTT**	carte satisfaction touristique, tendances e-commerce, alertes réputation

1.2 Architecture End-to-End

📄 **Architecture**

```
[ SOURCES ]  ──▶  [ NiFi ]  ──▶  [ KAFKA ]  ──▶  [ SPARK NLP ]  ──▶  [ STORAGE ]
 Avis/Reviews    Orchestration   Topic:           Sentiment          HBase/HDFS
 DGTT arrivées   & Routage       teranga_avis     Score NLP          Gold Layer
 Jumia/E-comm                    teranga_arrivees Prédiction flux          │
                                                        │            [ ANALYTICS ]
                                                        ▼             Apache Hive
                                              [ ALERT LAYER ]       vue_destinations
                                              Réputation ROUGE      vue_ecommerce
```

2. Prérequis & Infrastructure

Configuration minimale

RAM : 16 Go. CPU : 4 cœurs. OS : Ubuntu 22.04 LTS. Docker Engine 24.x. Java 11 requis par Spark.

setup.sh

```
# Démarrage infrastructure
mkdir -p data dags models nifi_templates
docker compose up -d zookeeper && sleep 10
docker compose up -d kafka nifi hbase hive-metastore && sleep 30
docker compose up -d spark-master spark-worker airflow

# Initialiser HBase (une seule fois)
python hbase_setup.py

# Démarrer le simulateur
python kafka_producer_teranga_sn.py &

# Interfaces web :
#   NiFi    : http://localhost:8081
#   Spark   : http://localhost:8080
#   HBase   : http://localhost:16010
#   Airflow : http://localhost:8082
```

requirements.txt

```
# requirements.txt — Teranga-SN
pyspark==3.3.2
kafka-python==2.0.2
pandera==0.17.2
pandas==2.0.3
apache-airflow==2.7.3
mlflow==2.8.1
pyhive==0.7.0
happybase==1.2.0
matplotlib==3.7.3
seaborn==0.12.2
faker==30.1.0
scipy==1.11.4
numpy==1.24.4
```

3. Codes Sources Complets & Fonctionnels

3.1 Initialisation HBase — hbase_setup.py

> ⚠️ **Fichier obligatoire**
> Crée les 3 tables HBase de Teranga-SN. À exécuter avant le démarrage du pipeline.

📄 **hbase_setup.py**

```
# hbase_setup.py — Tables HBase Teranga-SN
import happybase, logging
logger = logging.getLogger('HBaseSetup')

def create_teranga_sn_tables():
    conn = happybase.Connection('hbase', port=9090, timeout=10000)
    conn.open()
    tables = {
        b'teranga:avis_analyses': {'b'meta': {'max_versions': 1}},  # Avis avec score
sentiment (TTL 90j)
        b'teranga:alertes_reputation': {'b'meta': {'max_versions': 1}},  # Alertes
réputation destination (TTL 7j)
        b'teranga:stats_ecommerce': {'b'meta': {'max_versions': 1}},  # KPIs e-commerce
par catégorie
    }
    existantes = [t.decode() for t in conn.tables()]
    for nom_b, fam in tables.items():
        nom = nom_b.decode()
        if nom not in existantes:
            conn.create_table(nom_b, fam)
            logger.info(f'Table {nom} créée ✓')
    conn.close()

if __name__ == '__main__':
    logging.basicConfig(level=logging.INFO)
    create_teranga_sn_tables()
```

3.2 Simulateur de Données — kafka_producer

> ℹ️ **Données simulées réalistes**
> Le simulateur génère des données calées sur les réalités de Tourisme & E-commerce Local au Sénégal. Les champs PII (noms, IDs) sont présents dans le simulateur et seront anonymisés par la Privacy Layer Spark.

📄 **kafka_producer_teranga_sn.py**

```
# kafka_producer_teranga.py — Simulateur avis touristiques + e-commerce
from kafka import KafkaProducer
import json, random, time, uuid
from datetime import datetime
import numpy as np

random.seed(42); np.random.seed(42)
DESTINATIONS =
['DAKAR','SAINT_LOUIS','SALY','CAP_SKIRRING','CASAMANCE','TOUBA','ZIGUINCHOR']
TYPES_ACTIVITE = ['PLAGE','CULTURE','GASTRONOMIE','ARTISANAT','ECOTOURISME','PELERINAGE']
NATIONALITES   = ['SN','FR','MA','CI','GN','US','DE','IT','ES','GB']
CATEGORIES_ECOMM = ['ALIMENTATION','MODE','ELECTRONIQUE','ARTISANAT','TELECOM','BEAUTE']

# Avis positifs/négatifs typiques par langue
AVIS_POSITIFS = {
    'FR': ['Excellent séjour, accueil chaleureux teranga !',
```

```
            'Magnifique destination, cuisine délicieuse thiéboudienne',
            'Plage de Saly superbe, personnel très sympa',
            'Saint-Louis patrimoine UNESCO, incontournable'],
    'EN': ['Amazing hospitality, highly recommend Dakar',
            'Beautiful beaches, wonderful people',
            'Best trip ever to Casamance, paradise'],
    'WO': ['Dafa baax torop, teranga bi rafet na',  # C est très bon
            'Dakar dafa yomb, rafet na'],
}
AVIS_NEGATIFS = {
    'FR': ['Trop de déchets sur la plage, décevant',
            'Arnaque taxi, prix non affiché',
            'Hôtel pas propre, ne recommande pas'],
    'EN': ['Too many touts, not pleasant',
            'Overpriced for quality, disappointed'],
    'WO': ['Dafa neka, dafa cher na torop'],
}

producer = KafkaProducer(bootstrap_servers=['kafka:9092'],
    value_serializer=lambda v: json.dumps(v, ensure_ascii=False).encode())

def gen_avis():
    dest    = random.choice(DESTINATIONS)
    m       = datetime.utcnow().month
    # Haute saison nov-mars : plus de touristes et plus d'avis positifs
    prob_pos = 0.65 if m in [11,12,1,2,3] else 0.45
    langue  = random.choices(['FR','EN','WO'], weights=[60,30,10])[0]
    if random.random() < prob_pos:
        texte = random.choice(AVIS_POSITIFS.get(langue, AVIS_POSITIFS['FR']))
        note  = round(random.uniform(3.5, 5.0), 1)
    else:
        texte = random.choice(AVIS_NEGATIFS.get(langue, AVIS_NEGATIFS['FR']))
        note  = round(random.uniform(1.0, 3.0), 1)
    return {
        'avis_id':        str(uuid.uuid4()),
        'user_id':        f'USR_{uuid.uuid4().hex[:10]}',   # Sera anonymisé
        'email_client':   f'user{random.randint(1,9999)}@test.sn',  # Supprimé
        'destination':    dest,
        'type_activite':  random.choice(TYPES_ACTIVITE),
        'nationalite':    random.choice(NATIONALITES),
        'note':           note,
        'texte_avis':     texte,
        'langue':         langue,
        'plateforme':     random.choice(['TRIPADVISOR','GOOGLE','BOOKING','EXPAT_DAKAR']),
        'timestamp':      datetime.utcnow().strftime('%Y-%m-%dT%H:%M:%SZ'),
    }

def gen_ecommerce():
    return {
        'transaction_id': str(uuid.uuid4()),
        'user_id':        f'USR_{uuid.uuid4().hex[:10]}',
        'categorie':      random.choice(CATEGORIES_ECOMM),
        'montant_fcfa':   round(random.uniform(2000, 150000), 0),
        'region_livraison': random.choice(DESTINATIONS),
        'plateforme':     random.choice(['JUMIA','EXPAT_DAKAR','SENMARKET']),
        'timestamp':      datetime.utcnow().strftime('%Y-%m-%dT%H:%M:%SZ'),
    }

if __name__ == '__main__':
    print('Simulateur Teranga-SN démarré...')
    while True:
        producer.send('teranga_avis_raw', gen_avis())
        producer.send('teranga_arrivees_raw', gen_ecommerce())
        producer.flush(); time.sleep(3)
```

3.3 Pipeline Spark Streaming — streaming

> ⚠️ **Privacy critique**
> Drop des champs PII après anonymisation SHA-256. Les données sensibles ne doivent jamais apparaître dans HBase ou Hive.

streaming_teranga_sn.py

```
# streaming_teranga.py — Pipeline Spark NLP Teranga-SN
# Sentiment avis FR/EN/WO + prédiction réputation destination
import os
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, sha2, concat, lit, from_json, to_json, struct,
    window, avg, count, when, coalesce, lower, udf, current_timestamp
)
from pyspark.sql.types import *

SALT = os.environ.get('TERANGA_SECRET_SALT','default_salt')
spark = SparkSession.builder.appName('Teranga_SN_NLP') \
    .config('spark.sql.shuffle.partitions','4').getOrCreate()
spark.sparkContext.setLogLevel('WARN')

# Lexique sentiment FR/EN/Wolof tourisme
LEXIQUE = {
    'magnifique':0.9,'excellent':0.9,'superbe':0.85,'teranga':0.8,'baax':0.9,
    'dafa baax':0.9,'rafet':0.8,'recommend':0.8,'amazing':0.9,'wonderful':0.85,
    'décevant':-0.7,'arnaque':-0.9,'sale':-0.8,'cher':-0.5,'dafa neka':-0.8,
    'disappointed':-0.7,'overpriced':-0.6,'touts':-0.5,'déchets':-0.7,
}

@udf(returnType=FloatType())
def score_avis(texte):
    if not texte: return 0.0
    t = texte.lower(); score = 0.0; n = 0
    for terme, val in LEXIQUE.items():
        if terme in t: score += val; n += 1
    return float(score / max(n,1))

avis_schema = StructType([
    StructField('avis_id',       StringType(), True),
    StructField('user_id',       StringType(), True),
    StructField('email_client',  StringType(), True),
    StructField('destination',   StringType(), True),
    StructField('type_activite', StringType(), True),
    StructField('nationalite',   StringType(), True),
    StructField('note',          FloatType(),  True),
    StructField('texte_avis',    StringType(), True),
    StructField('langue',        StringType(), True),
    StructField('plateforme',    StringType(), True),
    StructField('timestamp',     StringType(), True),
])

avis_df = (spark.readStream.format('kafka')
    .option('kafka.bootstrap.servers','kafka:9092')
    .option('subscribe','teranga_avis_raw').load()
    .select(from_json(col('value').cast('string'), avis_schema).alias('d')).select('d.*')
    .withColumn('event_ts', current_timestamp())
    .withColumn('user_secure', sha2(concat(col('user_id'),lit(SALT)),256))
    .drop('user_id','email_client')
    .withColumn('sentiment_score', score_avis(col('texte_avis')))
    .withColumn('sentiment_label',
        when(col('sentiment_score') > 0.3, lit('POSITIF'))
        .when(col('sentiment_score') < -0.3, lit('NEGATIF'))
        .otherwise(lit('NEUTRE')))
)

# Agrégation réputation destination (fenêtre 7j)
reputation_df = (avis_df
    .withWatermark('event_ts','1 day')
```

```
    .groupBy(window('event_ts','7 days','1 day'), 'destination')
    .agg(
        avg(coalesce('note',lit(3.0))).alias('note_moy'),
        avg('sentiment_score').alias('sentiment_moy'),
        count('avis_id').alias('nb_avis'),
    )
    .withColumn('statut_reputation',
        when(col('sentiment_moy') < -0.3, lit('ROUGE'))
        .when(col('sentiment_moy') < 0.0,  lit('ORANGE'))
        .otherwise(lit('VERT')))
)

q1 = (reputation_df.select(to_json(struct('*')).alias('value'))
    .writeStream.format('kafka')
    .option('kafka.bootstrap.servers','kafka:9092')
    .option('topic','teranga_alerts')
    .option('checkpointLocation','/tmp/teranga_rep_ckpt')
    .outputMode('update').start())
q1.awaitTermination()
```

3.4 Tables Hive & Vues Analytics — hive_setup.sql

Les vues Hive utilisent systématiquement COALESCE sur les colonnes nullables pour éviter les NULL silencieux dans les agrégations.

hive_setup.sql

```
-- hive_setup.sql — Tables Hive Teranga-SN
CREATE DATABASE IF NOT EXISTS teranga_sn COMMENT 'Tourisme Sénégal — UADB 2025';
USE teranga_sn;

CREATE TABLE IF NOT EXISTS avis_analyses (
    avis_id STRING, user_secure STRING,
    destination STRING, type_activite STRING,
    nationalite STRING, note DOUBLE,
    sentiment_score DOUBLE, sentiment_label STRING,
    langue STRING, plateforme STRING, ingestion_ts TIMESTAMP
) PARTITIONED BY (date_obs STRING)
STORED AS ORC TBLPROPERTIES ('orc.compress'='SNAPPY');

CREATE TABLE IF NOT EXISTS ecommerce_transactions (
    user_secure STRING, categorie STRING,
    montant_fcfa DOUBLE, region_livraison STRING,
    plateforme STRING, ingestion_ts TIMESTAMP
) STORED AS ORC;

-- Vue : Satisfaction touristique par destination
CREATE OR REPLACE VIEW vue_destinations AS
SELECT destination,
    COUNT(avis_id)                  AS nb_avis_total,
    AVG(COALESCE(note,3.0))         AS note_moy,
    AVG(COALESCE(sentiment_score,0)) AS sentiment_moy,
    SUM(CASE WHEN sentiment_label='POSITIF' THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS
pct_positif,
    SUM(CASE WHEN sentiment_label='NEGATIF' THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS
pct_negatif,
    CASE
        WHEN AVG(sentiment_score) < -0.3 THEN 'ROUGE'
        WHEN AVG(sentiment_score) < 0.0  THEN 'ORANGE'
        ELSE 'VERT'
    END AS statut_reputation
FROM avis_analyses
WHERE date_obs >= DATE_SUB(CURRENT_DATE(),30)
GROUP BY destination ORDER BY note_moy DESC;

-- Vue : Top catégories e-commerce par région
CREATE OR REPLACE VIEW vue_ecommerce AS
SELECT region_livraison, categorie,
    COUNT(*)              AS nb_transactions,
    SUM(montant_fcfa)     AS ca_total_fcfa,
    AVG(montant_fcfa)     AS panier_moyen_fcfa
FROM ecommerce_transactions
GROUP BY region_livraison, categorie
ORDER BY ca_total_fcfa DESC;
```

3.5 DAG Airflow MLOps

Le BranchPythonOperator détecte automatiquement les dérives et déclenche le réentraînement du modèle si nécessaire. Les recommandations sont écrites dans HBase pour un accès temps réel.

dags/teranga_sn_dag.py

```
# dags/teranga_sn_dag.py — DAG Airflow Teranga-SN
from airflow import DAG
from airflow.operators.python import PythonOperator, BranchPythonOperator
```

```
from airflow.operators.dummy  import DummyOperator
from airflow.utils.dates      import days_ago
from datetime import timedelta
import subprocess, logging

logger = logging.getLogger('teranga_dag')
default_args = {'owner':'seye_ahmed','retries':2,'retry_delay':timedelta(minutes=5)}

def check_reputation(**ctx):
    from pyhive import hive
    conn = hive.Connection(host='hive-metastore',port=10000)
    cur  = conn.cursor()
    cur.execute("SELECT COUNT(*) FROM teranga_sn.vue_destinations WHERE
statut_reputation='ROUGE'")
    n = cur.fetchone()[0]
    logger.info(f'Destinations en ROUGE : {n}')
    return 'retrain_model' if n > 1 else 'update_dashboard'

def retrain_model(**ctx):
    subprocess.run(['spark-submit','--master','spark://spark-master:7077',
        '/opt/models/train_flux_model.py'],check=True,timeout=3600)

def update_dashboard(**ctx):
    import happybase; from pyhive import hive; from datetime import datetime
    conn_hbase = happybase.Connection('hbase',port=9090); conn_hbase.open()
    conn_hive  = hive.Connection(host='hive-metastore',port=10000)
    cur = conn_hive.cursor()
    cur.execute('SELECT destination,nb_avis_total,note_moy,sentiment_moy,statut_reputation
FROM teranga_sn.vue_destinations')
    table = conn_hbase.table(b'teranga:alertes_reputation')
    for row in cur.fetchall():
        dest,nb,note,sent,statut = row
        if statut != 'VERT':
            table.put(dest.encode(),{
                b'alerte:destination':dest.encode(),
b'alerte:note':str(round(note,2)).encode(),
                b'alerte:sentiment':str(round(sent,3)).encode(),
b'alerte:statut':statut.encode(),
                b'alerte:ts':datetime.utcnow().isoformat().encode()})
    conn_hbase.close()

with DAG('teranga_sn_monitoring', default_args=default_args,
         schedule_interval='0 7 * * 1', start_date=days_ago(1), catchup=False,
         tags=['teranga-sn','tourisme','ecommerce']) as dag:
    start  = DummyOperator(task_id='start'); end = DummyOperator(task_id='end')
    branch = BranchPythonOperator(task_id='check_reputation',
                                  python_callable=check_reputation,provide_context=True)
    t_train=
PythonOperator(task_id='retrain_model',python_callable=retrain_model,provide_context=True)
    t_upd  =
PythonOperator(task_id='update_dashboard',python_callable=update_dashboard,provide_context
=True)
    start >> branch >> [t_train, t_upd]
    t_train >> t_upd >> end; t_upd >> end
```

4. Livrables Attendus

#	Livrable	Contenu attendu
1	**Pipeline de Données**	Code source complet (GitHub) + template NiFi. Données visibles dans HDFS, topic Kafka teranga_avis_raw actif.
2	**Modèle ML & MLOps**	Script d'entraînement + RMSE/R^2 documentés + DAG Airflow avec BranchPythonOperator fonctionnel.
3	**Catalogue Hive**	Tables créées, vues fonctionnelles avec au moins 1 alerte visible dans HBase en démonstration.
4	**Dashboard & Recommandations**	Dashboard graphique (Matplotlib) + recommandations concrètes contextualisées au Tourisme & E-commerce Local sénégalais.

5. Barème de Notation — /20 points

> ℹ️ **Barème différencié 4 niveaux**
> Chaque critère est évalué selon 4 niveaux de maîtrise. La démonstration live en soutenance est obligatoire. Bonus documentation +2 pts.

Critère	Pts	Insuffisant (0)	Passable (50%)	Bien (75%)	Excellent (100%)
Ingestion NiFi Multi-Sources	**3 pts**	NiFi absent	1 source/3	Avis + DGTT	3 sources (avis+DGTT+Jumia), 2 topics distincts, messages en FR/EN/WO
Validation & Privacy (Transactions)	**4 pts**	Aucune validation, user_id en clair	Schéma partiel	SHA-256 sans drop user_id	Pandera + drop(user_id, email_client) + 0 PII données e-commerce dans HBase
NLP Sentiment & Streaming	**6 pts**	Pas de Spark	Lecture seule Kafka	Score lexical basique	Lexique FR/EN/WO (teranga, baax, magnifique) + 2 queries + watermarks
Prédiction Flux & MLOps	**4 pts**	Aucun modèle	Règles fixes saison	RF entraîné sans DAG	RF prédiction flux + DAG hebdomadaire + RMSE documenté + MLflow
Dashboard DGTT & Réputation	**3 pts**	Aucun dashboard	KPIs vides	Carte sentiment sans alertes	Carte réputation destinations + alertes ROUGE + top produits e-commerce
TOTAL	**20 pts**				

Bonus Documentation (jusqu'à +2 pts)

Élément	Points bonus
README.md avec docker compose up + captures d'écran	+0.5 pt
Commentaires de code en français + docstrings	+0.5 pt
Tests pytest sur les schémas Pandera et les UDFs	+0.5 pt
Démo live d'une alerte HBase sans erreur en soutenance	+0.5 pt

🌍 **Bon courage pour Teranga-SN !**

"Always be a solution, never a problem."

Mr Ahmed Ben Sidy Bouya SEYE | Senior Big Data & AI Engineer | Groupe Sonatel